Field Hours

CSCE 678 Distributed Systems and Cloud Computing

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 90 minutes

Partial failure and a record

You design a job so the death of 1 machine out of 50 does not cancel the other 49. The record of work is replicated, not kept in one process. A worker writes output under a name that fixes the bytes, then appends a completion record. If it dies between those 2 steps, another worker may redo the task. Doing it twice still leaves 1 result, because the name determines the bytes.

You treat a slow machine and a dead machine as the same suspicion until a record proves otherwise. A heartbeat every 2 seconds, missed 3 times, is 6 seconds of silence, after which the scheduler may reassign the task. The first worker can be alive and writing. Partial failure means some messages arrived and some did not, and you will not get a stack trace from the missing half. The completion record is what the rest of the job trusts.

Local memory dies with the worker. The completion record is what the other 49 machines can see.

Worker memory

Dies with the process

May sit between 2 steps

Not visible to the job

Shared record

Name fixes the bytes

Completion is a record

A redo leaves 1 result

A worker dies between two steps

1. The worker writes the output under a determined name, then should append the completion record. It dies in between.

2. After 3 missed heartbeats, 2 seconds apart, the scheduler waits 6 seconds and reassigns the task.

3. The second worker writes the same bytes under the same name. You keep 1 result, not 2.

Problem 1. Heartbeats are every 2 seconds and you reassign after 3 misses. How many seconds of silence is that?

Answer. 6 seconds. The first worker may still be alive. The output name must determine the bytes so a

redo does not create a second result.

Problem 2. Why does running the same task 2 times leave 1 result?

Answer. The output name fixes the bytes. The second run writes the same place. A completion record, not the worker's memory, is what the rest of the job trusts.

Lecture 2 · 90 minutes

Map, shuffle, and a backup

You express a batch job as a map and a reduce. Each map reads one split and emits pairs. With 4 mappers and 400 megabytes of input, each map sees about 100 megabytes. The shuffle groups pairs by key and gives one key to one reducer. A reducer for key 7 sees every pair for that key, from all 4 maps, and writes one output. You do not give key 7 to 2 reducers.

You recompute a split instead of patching a half-written file. If mapper 2 of 4 dies, you run its split again. In the model the shuffle waits for all 4 map outputs before a reducer starts. A straggler that is 5 times slower holds the job, so you launch a backup copy and keep the output that finishes first. The backup is safe only because the map is deterministic: one split yields one set of pairs.

One key goes to one reducer. A second copy of a map is safe only if the split is deterministic.

Map 4 splits -> Shuffle by key -> Reduce each key once -> Backup a slow map

Four maps and one slow split

1. Input is 400 megabytes. Four mappers each read about 100 megabytes and emit pairs.

2. Key 7 from all 4 maps arrives at one reducer. A second reducer does not also own key 7.

3. Mapper 2 is 5 times slower than the others. You start a backup copy and keep whichever output finishes first. Both copies must emit the same pairs.

Problem 1. Four mappers read 400 megabytes, split evenly. About how much does each map read, and may two reducers both own key 7?

Answer. About 100 megabytes each. No. The shuffle gives key 7 to one reducer so that reducer sees every pair for that key.

Problem 2. Mapper 2 of 4 is 5 times slower and you start a second copy. Why is keeping the first finished output safe?

Answer. The map is deterministic: one split yields one set of pairs. The copy writes the same output. You recompute instead of patching a half-written file.

Lecture 3 · 90 minutes

Counters and leases

You cannot treat wall clocks on 2 machines as one clock. A Lamport counter advances on each local event. If you receive a stamp of 5 while your counter is 3, you set it to 6 before work. When this rule orders event a before b, the stamp of a is smaller. A smaller stamp does not prove earlier wall time. You use the stamps to order a log,

not to measure the 6 seconds a heartbeat might suggest.

You grant a lock as a lease of 10 seconds, not until the owner calls unlock. If the owner dies, another worker may take the lock after expiry. A write at second 11 is late, and you reject it. The expiry is checked on every use. You renew near second 5 if the work continues. Waiting forever for an unlock waits forever after 1 crash.

The counter orders events. The lease ends whether or not the holder calls unlock.

1. Receive stamp 5

2. Local counter was 3

3. Set the counter to 6

4. Reject a write at second 11

Stamp 5 meets counter 3

1. Your counter is 3. A message arrives stamped 5. You set the counter to 6 before the next local event.

2. That orders the receive after the send. It does not say the send happened 6 seconds earlier on a wall clock.

3. A lock leased for 10 seconds is a different clock: a write at second 11 is rejected even if the holder never crashed, only slowed.

Problem 1. Your Lamport counter is 3 and you receive a message stamped 5. What do you set the counter to?

Answer. 6. It must be at least one past the stamp you received and past your old value. A smaller stamp does not prove earlier wall-clock time.

Problem 2. A lock lease is 10 seconds. The holder writes at second 11 and never called unlock. Do you accept the write?

Answer. No. The lease expired, so the holder no longer owns the lock. You reject the write. Renew near second 5 if the work should continue.

Lecture 4 · 90 minutes

A ring and a finger

You look up a key on a peer ring without a central directory. Each node stores its successor, the next id clockwise. A query walks successors until the node whose arc contains the key, and on a ring of 8 nodes that walk is up to 7 hops. A finger table cuts the hops. From a node, the fingers reach distances 1, 2, and 4, so one hop can skip half the remaining ids instead of stepping one successor.

You store a key on the successor of its id. To survive 2 deaths you also copy it to the next 2 nodes. Finger lookup on 8 nodes takes about 3 hops, since a hop can halve the distance. A joining node takes the keys now on its arc from its successor. You do not notify all 8 nodes. Lagging fingers still end at the right successor.

Successors alone are a long walk. Fingers skip ahead by doubling distances.

8 nodes on one ring

Successor is the next id

Fingers at 1, 2, and 4

Key copied to 2 more nodes

Lookup on eight nodes

1. Walking only successors on 8 nodes takes up to 7 hops to reach the arc that owns the key.

2. Fingers at distances 1, 2, and 4 let a hop halve the remaining distance. The lookup is about 3 hops.

3. The key lives on the successor of its id, plus the next 2 nodes, so 2 deaths do not erase it. A join moves only the keys on the new arc.

Problem 1. A ring has 8 nodes and you walk only successors. What is the worst-case hop count, and about how many hops do fingers take?

Answer. Up to 7 hops along successors. Fingers, at distances 1, 2, and 4, bring it to about 3 hops by halving the remaining distance.

Problem 2. You want the key to survive 2 deaths. How many extra nodes, beyond the successor, hold a copy?

Answer. 2. The key is on the successor and the next 2 nodes. A joining node takes only the keys that fall on its new arc. You do not notify all 8 nodes.

Lecture 5 · 90 minutes

A majority before you answer

You elect a leader before appending to a replicated log. Servers keep a term number. After 150 milliseconds of silence a follower becomes a candidate, adds 1 to its term, and requests votes. With 5 servers the majority is 3, including its own vote. A split of 2 and 2, plus 1 server that has not voted, elects nobody. They wait a random delay and open another term.

You commit a log entry only after a majority has stored it. The leader appends the entry, sends it to the other 4 servers, and waits until 2 of them store it, which makes 3 copies. Then it may answer the client. If you answer after only 1 copy, a crash loses an entry you already acknowledged. An old leader cannot overwrite a newer majority. The 3-of-5 rule is the agreement, not a hint about speed.

Votes pick a leader. Copies of the entry, not the election itself, are what let you answer.

Silence for 150 ms -> Collect 3 votes of 5 -> Store the entry on 3 -> Then answer the client

Three copies, then the reply

1. Five servers, majority 3. The candidate counts itself and needs 2 more votes.

2. As leader it appends an entry and must see 2 other servers store it, for 3 copies, before it answers the client.

3. Answering after the local copy alone means a crash can drop an entry the client was told was committed. A 2-2 vote split elects nobody and starts another term after a random delay.

Problem 1. There are 5 servers. How many votes elect a leader, and how many stored copies must an entry have before you answer the client?

Answer. 3 votes, counting the candidate. The entry needs 3 stored copies: the leader plus 2 of the other 4. Answering after 1 copy can lose an acknowledged entry.

Problem 2. Two candidates split the remaining votes 2 and 2, and 1 server has not voted. Who leads?

Answer. Nobody. Neither candidate has 3 votes. They wait a random delay and open another term. A majority of 3 is the election, not a plurality.

Lecture 6 · 90 minutes

Quorums that intersect

You size quorums so every read meets every write. On 3 replicas, write 2 and read 2 intersect in at least 1. Tag each write, and the reader keeps the highest tag in its quorum. Read count plus write count must exceed 3: here 2 + 2 does. If both quorums are 1, they need not intersect, and a read can miss a write for good.

You can send every write to 1 primary and read that primary, which orders updates without comparing tags. Quorum writes stay up when any 1 of the 3 replicas is down, because 2 remain and a write quorum is 2. They stop when 2 are down, because you cannot gather those 2. You choose quorums when any replica may accept a write. You choose a primary when you want one order and you can stand losing that one machine.

Intersection is the whole rule. A primary is a different way to get one order.

Quorum 2 of 3

Read 2 and write 2

They share at least 1

Highest tag wins

One primary

Every write hits 1 primary

Reads can use that primary

Down primary stalls writes

One replica down

1. N is 3, write quorum is 2, read quorum is 2. Replica 3 is down.

2. A write still finds 2 live replicas, so it succeeds. Any later read of 2 replicas must include at least 1 of those writers.

3. If replicas 2 and 3 are both down, only 1 remains and the write quorum of 2 cannot form. Both quorums of 1 would have allowed a read to miss the write.

Problem 1. N is 3, R is 2, and W is 2. One replica is down. Can you write, and why must a later read see that write?

Answer. Yes, 2 replicas remain, which is the write quorum. Any read of 2 intersects any write of 2 in at least 1 replica, so the reader can take the highest tag.

Problem 2. Both quorums are 1 on 3 replicas. What can a read miss?

Answer. A write that landed on a different replica. The sets need not intersect, because 1 + 1 does not exceed 3. The reader can miss that write for good.

Lecture 7 · 90 minutes

Slots, shares, and stragglers

You place tasks on slots. One machine with 8 slots runs 8 tasks. A job with 4 maps and 2 reduces needs 6 slots if they overlap, or 4 then 2 if reduces wait. Fairness, with 2 users waiting, gives each 4 of the 8 slots. If A has 100 tasks and B has 2, a fair share still runs B. Filling all 8 with A makes B wait behind 100 tasks.

You copy a straggler. When 1 of 4 maps remains after the other 3 finish, you start a second copy on a free slot and keep the first output. If the disk on a free machine lacks the split, you wait about 1 second for a local slot, then run remotely. You do not wait forever for the local copy, and you do not skip the wait on every task.

Fair share is about who waits. Locality is a short wait, not a permanent one.

1. Count 8 free slots

2. Give 4 to each of 2 users

3. Copy the slow map

4. Wait 1 second for local data

Two users, eight slots

1. The machine has 8 slots. Users A and B are both waiting. Each receives 4 slots.

2. A has 100 tasks and B has 2. B's tasks run in B's 4 slots instead of waiting until A's 100 finish.

3. Separately, 1 of 4 maps is left after the other 3 finish. You start a copy on a free slot. If that slot's disk lacks the split, you wait about 1 second, then run remotely.

Problem 1. Eight slots and 2 waiting users. How many slots does a fair split give each, and what happens to B if A is allowed to fill all 8?

Answer. Each user gets 4. If A fills all 8, B waits behind A's tasks. With A holding 100 tasks and B holding 2, that wait is the failure of fairness.

Problem 2. Three of 4 maps have finished and 1 remains. What do you launch, and how long do you wait for a machine that already holds the split?

Answer. A second copy of the remaining map, and you keep the first output to finish. You wait about 1 second for a local slot, then you run remotely rather than waiting forever.

Lecture 8 · 90 minutes

A job you can specify

You can now specify a job that survives 1 dead worker and 1 dead leader. You run 4 mappers, shuffle by key, and reduce, and you copy a map that is 5 times slower. You order events with a counter that jumps to 6 on a stamp of 5. You take a lock only as a 10-second lease, and you commit a log entry only at 3 copies out of 5.

You can say which mechanism answers which failure. A backup map is safe because the map is deterministic and the same split yields the same pairs. A write quorum of 2 still succeeds when 1 of 3 replicas is down, and it fails when 2 are down. Fairness on 8 slots gives 4 slots to each of 2 waiting users. Answering a client after 1 log copy would make the majority rule false.

Match the failure to the record that survives it. A timeout alone is not that record.

Deterministic map, plus a copy

Majority of 3 out of 5

Quorum 2 when 1 is down

4 slots for each of 2 users

A slow map and one dead replica

1. One of 4 mappers is 5 times slower. You launch a second copy and keep the first finished output, which is safe because the split determines the pairs.

2. That output is written with a quorum of 2 out of 3. One replica is down, and 2 remain, so the write still forms.

3. If 2 replicas were down, the quorum could not form and the finished map would have nowhere to land under that rule. The copy and the quorum solve different failures.

Problem 1. One of 4 mappers is 5 times slower, and each map output is written to a quorum of 2 out of 3 replicas. Why is a second copy of the slow map safe, and can the write finish if 1 replica is down?

Answer. The copy is safe because the map is deterministic: the same split yields the same pairs, and you keep one finished output. A write quorum of 2 still succeeds when 1 of 3 replicas is down. If 2 are down, the quorum cannot form.

Problem 2. A leader has stored a log entry on 1 server and wants to answer the client. Separately, 2 users share 8 slots. What is wrong with the answer, and how many slots does each user get under a fair split?

Answer. The entry is not committed: with 5 servers you need 3 stored copies before you answer, and 1 copy can be lost. Each of the 2 users gets 4 of the 8 slots. The early answer and the slot split are different rules.